

ECC-uncorrectable メモリ エラーへの耐性を有する ページテーブル管理機構

2024/05/30

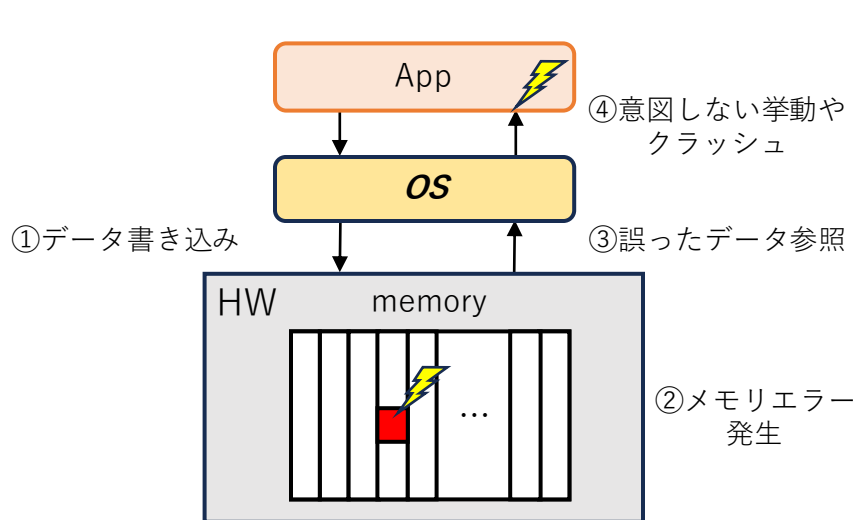
武田一希 山田浩史

東京農工大学

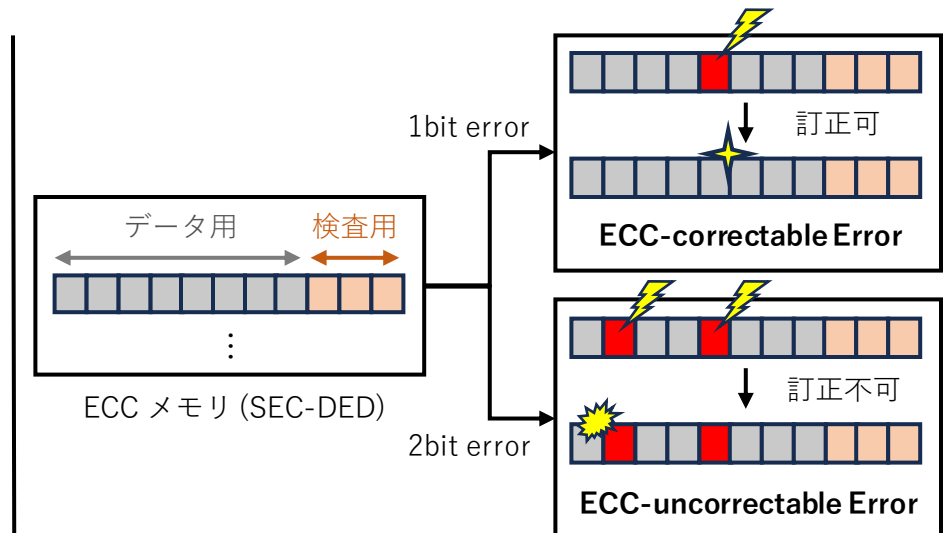
Memory Errors

2

- メモリセルが保持するデータを正しく読み出せなくなる現象
 - 宇宙線の衝突などによるビット反転, メモリセルの物理的破損により発生
 - 誤ったデータの参照により, 意図しない挙動やソフトウェアのクラッシュ
 - 実環境下でメモリエラーは頻繁に報告されている
 - データセンタでは一年間で 32% のサーバが経験 [Schroeder+, SIGMETRICS'09]
 - DRAM の微細化によりメモリエラーが増加している [Meza+, DSN'15]
 - 不揮発性メモリの 3D Xpoint はメモリエラーが発生しやすい [Zhang+, ISCA'16]
- Error-Correcting Code (ECC) : 検査用ビットにより誤りを検知・訂正
 - シングルエラー訂正・ダブルエラー検出 (SEC-DED) が一般的に使用されている



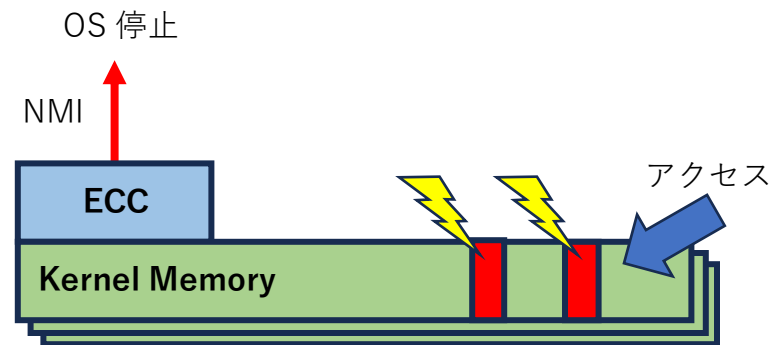
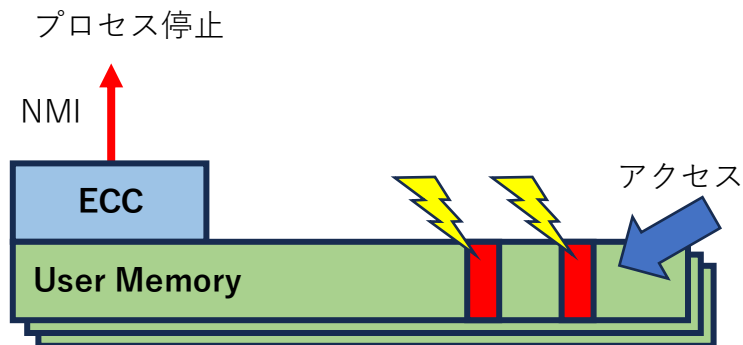
メモリエラーの概要



ECC メモリによる対策

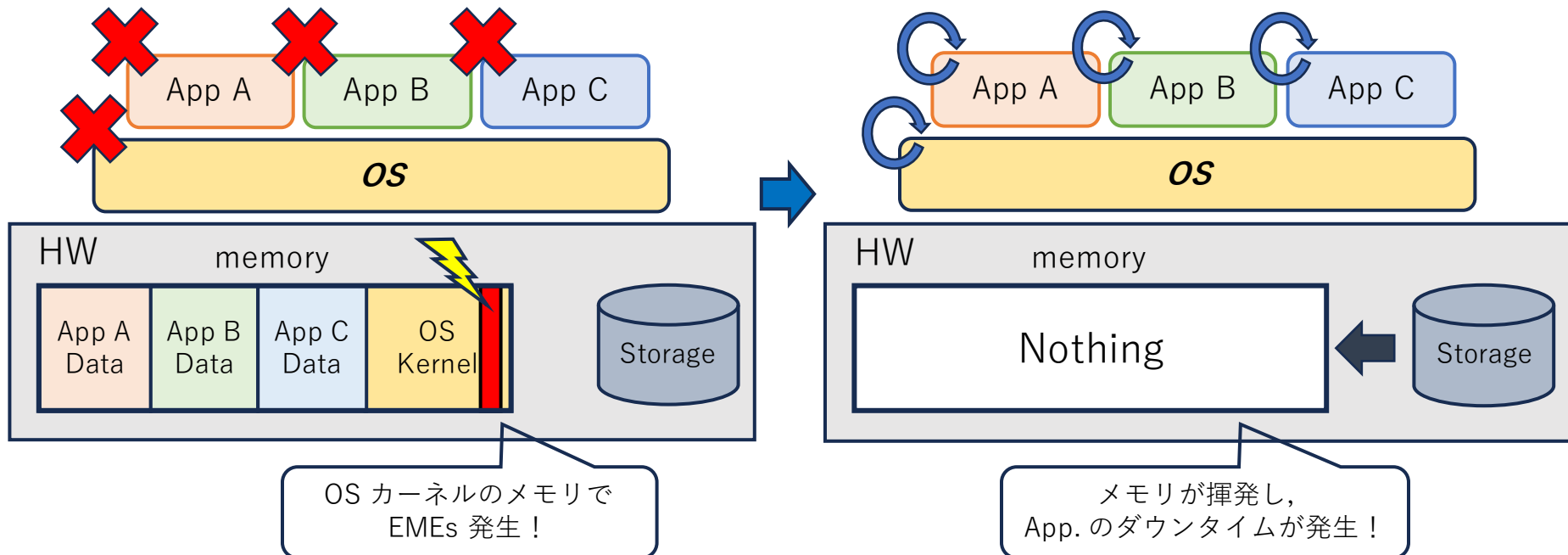
ECC-uncorrectable Memory Errors (EMEs) 3

- ECC モジュールにより検知可能・修正不可能なメモリエラー
 - EMEs が生じたメモリセルにアクセスすると検知
 - ECC モジュールが割り込み (NMI) を発生させてアドレスとともに OS に通知する
 - アドレスがユーザ空間 $\Rightarrow$ 該当プロセス停止 & カーネル空間 $\Rightarrow$ OS 停止, が一般的
- Real-world なエラー
 - 様々なフィールドスタディで EMEs の発生が報告されている [Maneas+, FAST'20], [Patel+, DSN'19], [Sridharan+, ASPLOS'15], etc.
 - e.g.) SSD 故障の 32.78% がドライブの DRAM 上の EMEs が原因 [Maneas+, FAST'20]
 - EMEs の対策手法が提案されている [Borchert+, DSN'23], [Iguchi+, ISSRE'22], [Shimomura+, DSN'22], [Du+, MEMSYS'21], [Zhang+, USENIX ATC'19], etc.
 - e.g.) Partial-Surgery[Shimomura+, DSN'22]: In-memory KVS を対象に, EMEs で壊れた KV やメタデータを破棄し, 内部構造を修正し継続動作を実現



EMEs in the OS Kernel

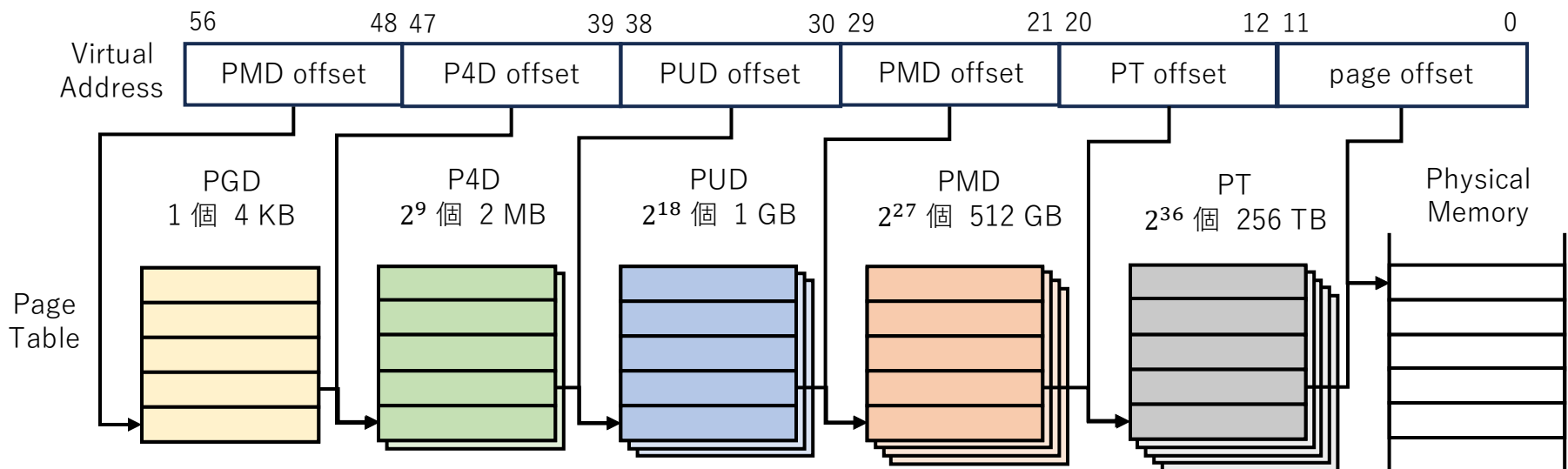
- 現行の OS カーネルでは全 App. を含めて再起動することで回復
 - 全 App. の処理が中断し、メモリ内容がすべて揮発する
 - メモリの再読み込みによる、App. サービスのダウンタイムが発生
 - 特に大量のメモリを要求する Memory-intensive な App. の場合、メモリ復元が長期化し、サービスの信頼性や可用性を損なう
 - e.g.) Facebook 社のデータセンタ内の 2% のサーバマシンの復旧に 12 時間費やし、その間是一部のクエリしか実行できない [Goel+, SIGMOD'14]
- ⇒ OS カーネルのメモリに EMEs への耐性を付与する必要がある



Memory Footprint of Page Tables

5

- ページテーブルはメモリフットプリントが大きいため、他のオブジェクトより相対的に EMEs が発生する可能性が高い
 - メモリの増加に伴い、x86 では 5 階層ページテーブルを使用
 - 最下位ページテーブルは最大で 2^{36} 個、メモリサイズは 256 TB
 - ユーザプロセス数が増加するほど、ページテーブルの数も増加
 - 特に Memory-intensive な App. の場合、ページテーブルは肥大化
 - e.g.) 2020 年 12 月時点で、ヤフーでは 204,980 個以上のコンテナが稼働
全コンテナが 4GB メモリ使用 $\Rightarrow$ ページテーブルは 1600 GB $\sim$ 800 TB
 - ページテーブルの 1 ページでも EMEs によりダメージを受けると App. は全滅
 $\Rightarrow$ ページテーブルに EMEs への耐性を付与する必要がある



- Intel Address Range Partial Memory Mirroring
 - 複製を用いて指定したメモリに EMEs への耐性を付与する手法
 - 複製可能なメモリサイズは 4 GB までという制限がある
 - 単純計算で 2 倍のメモリが必要であり、高メモリスペースオーバーヘッド
- Partial-surgery [Shimomura+, DSN'22], Ev6 [Iguchi+, ISSRE'22]
 - In-memory KVS や OS カーネルに EMEs への耐性を付与する手法
 - 実践的な OS のメモリ管理機構が対象ではない
- Elastic Cuckoo Page Tables [Skarlatos+, ASPLOS'20]
 - ページテーブルの構成を変更し、App. の性能を高める手法
 - ページテーブルサイズが小さくなる
 - ハードウェアの変更が必要

Proposal

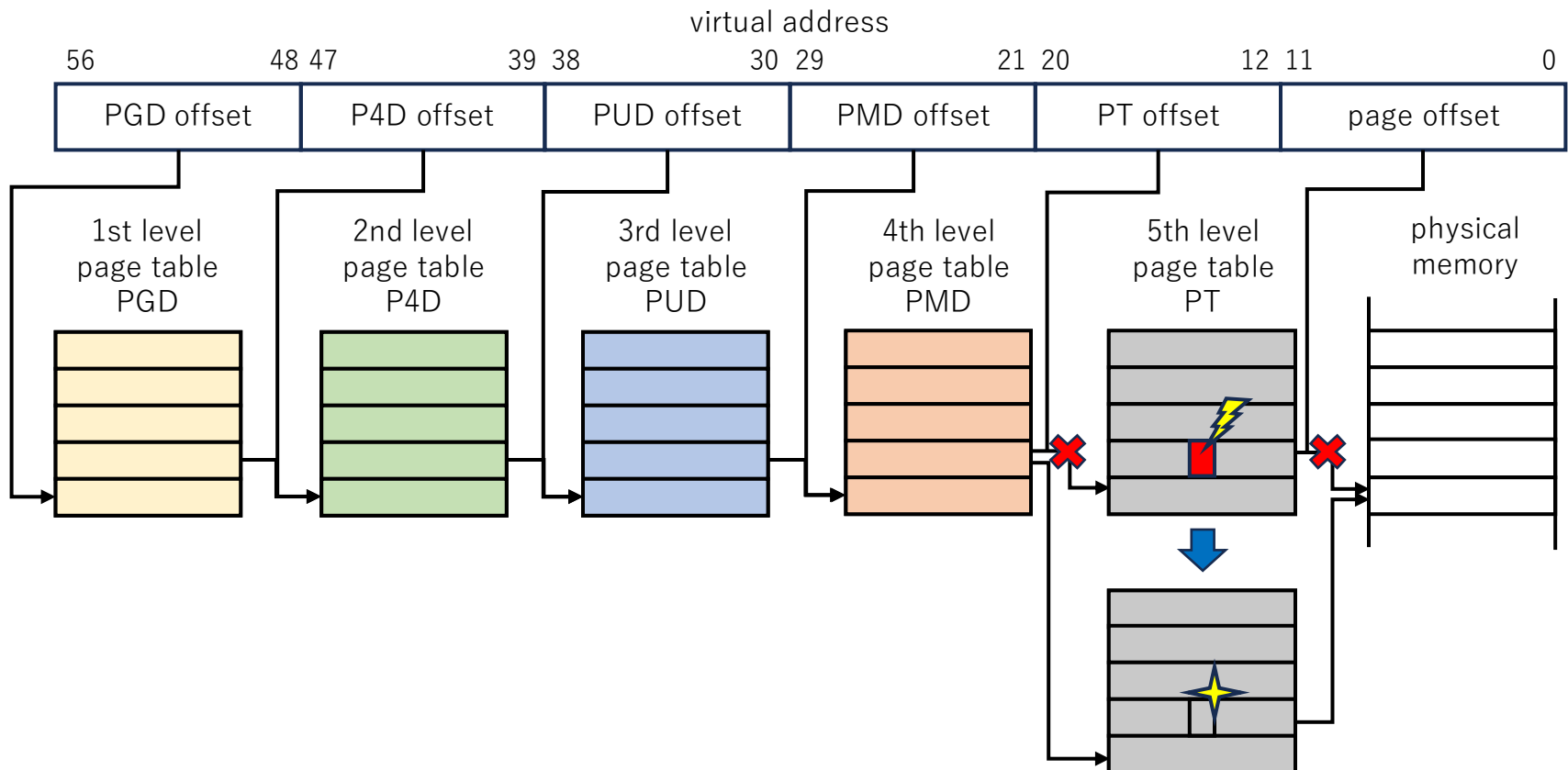
7

- EMEs への耐性を有するページテーブル管理機構
 - ページテーブルにて EMEs が発生しても、再起動を回避し、OS カーネルや App. を継続して使用可能
 - 低メモリスペースオーバヘッドで実現
 - 実践的な OS に適用可能
 - Case Study: Linux 6.1.35
 - ハードウェアの変更を必要としない

Approach

8

- 破損したページテーブルを 1 ページ単位で復元
 - あらかじめページテーブルの内容をデータ構造として保持し、内容を復元
 - エラーによる破損個所以外のメモリは健全であることに着目
 - 最もメモリフットプリントが大きい最下位ページテーブルでの EMEs を想定



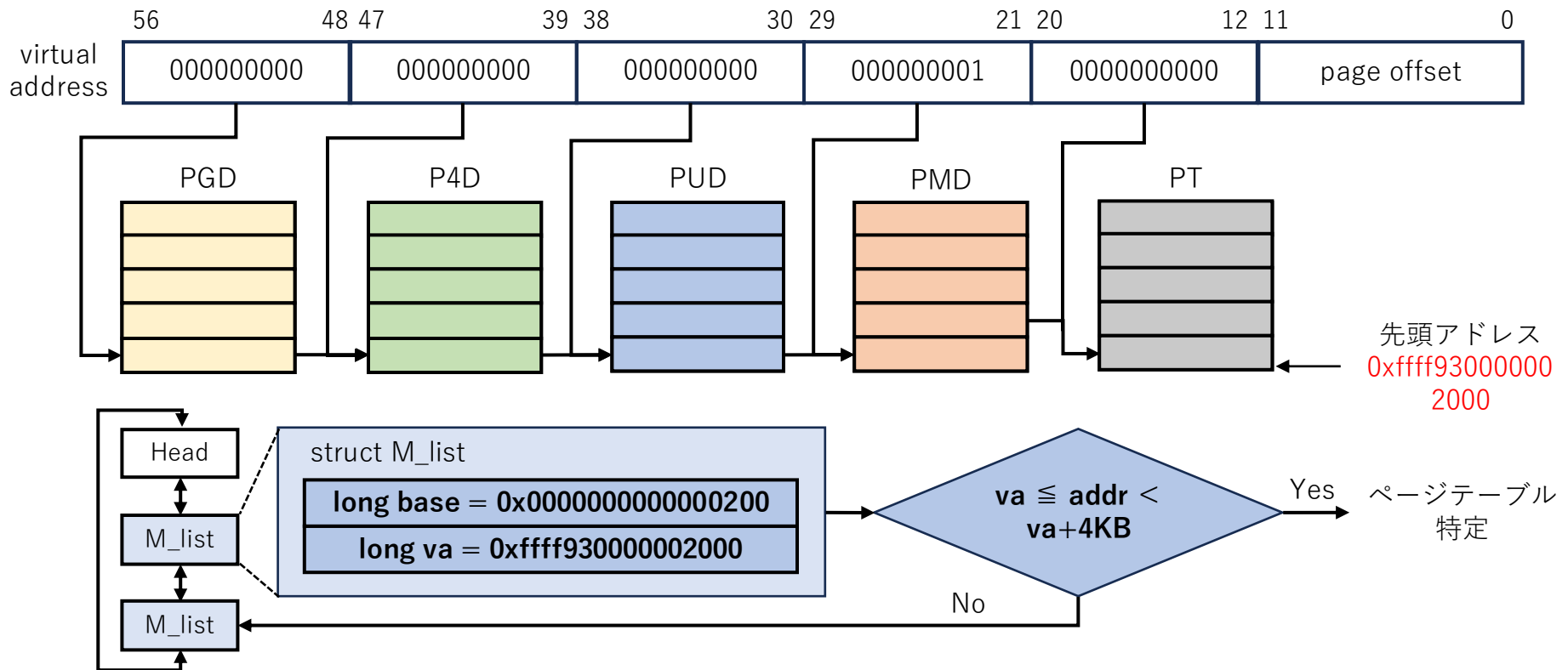
Design Challenges

- どのように破損したページテーブルを特定するのか？
 - NMI によって得られるのは破損個所のアドレスのみ
 - ⇒ アドレスからページテーブルを特定 (M_list)
- どのように低メモリスペースオーバーヘッドで内容を復元するのか？
 - 単純に最下位ページテーブルの複製を保持する場合,
2 倍のメモリが必要であり, 高メモリスペースオーバーヘッド
 - ⇒ 複数のページテーブルエントリの情報を圧縮して保持 (DS_list)
- マルチスレッドで同じページテーブルに更新・アクセスする際に EMEs が発生した場合でも適切に復元し, 処理を継続できるか？
 - スレッド 1 が PTE 1 更新中にスレッド 2 が PTE 2 読み込みにて EMEs を検知
 - スレッド 1 の PTE の更新が完了されていない状態で復元する可能性がある
 - スレッド 2 で取得したページテーブルのアドレスを新しいものに変更する必要がある
 - ⇒ ページテーブルの更新・アクセスをトランザクションでログを取る

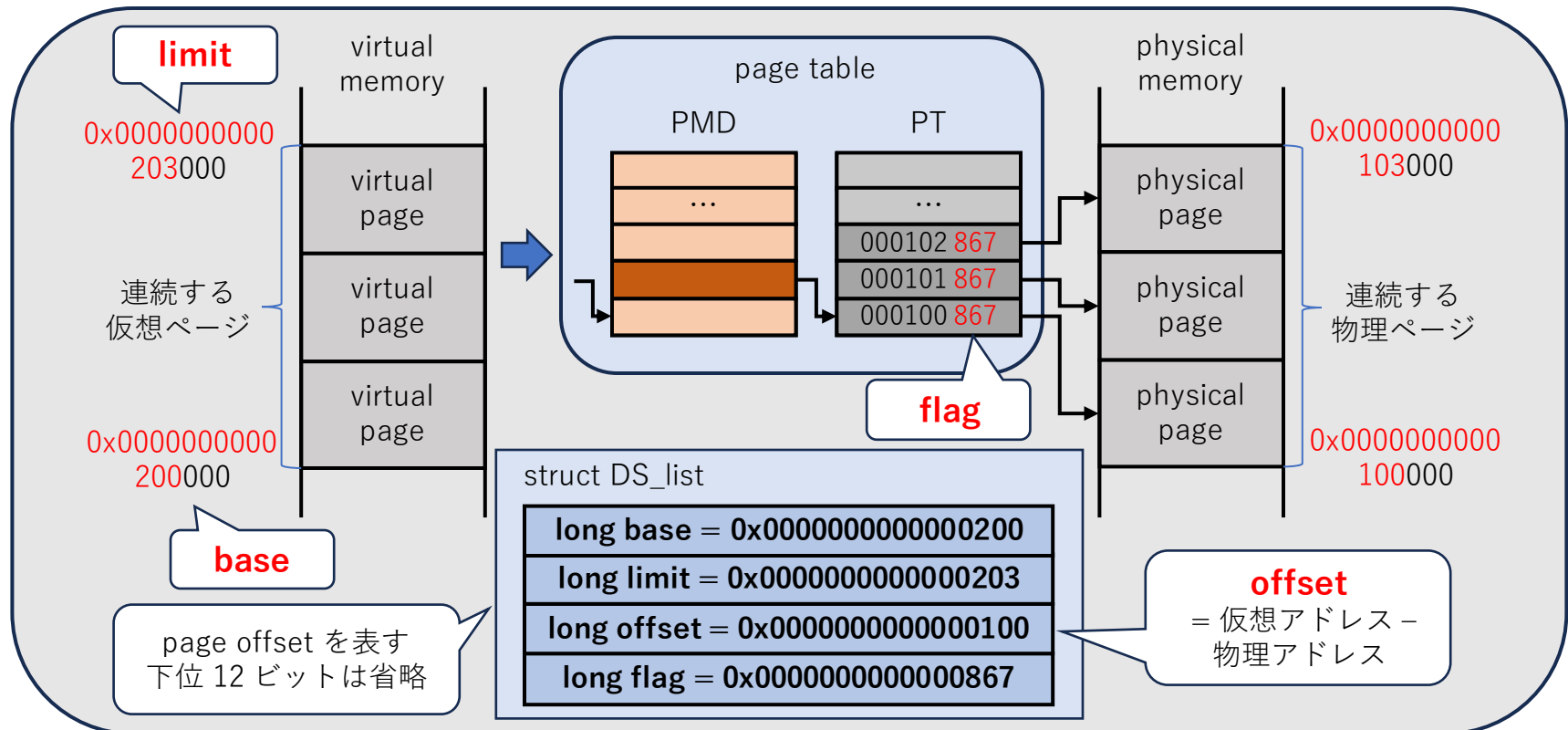
M_list

10

- 通知されたアドレスからページテーブルを特定するデータ構造
 - ページテーブル仮想先頭アドレス *VA*, ユーザ仮想先頭アドレス *BASE* の 2 つの値を保持するリスト構造体
 - ページテーブルのメモリを割り当てた際にノードを作成
 - 通知されたアドレスがページテーブル仮想先頭アドレス + 4KB 以内か判定
 - 真ならば, ページテーブルを特定することができ, ユーザ仮想先頭アドレスを取得
 - 偽ならば, 次の M_list のノードを探索



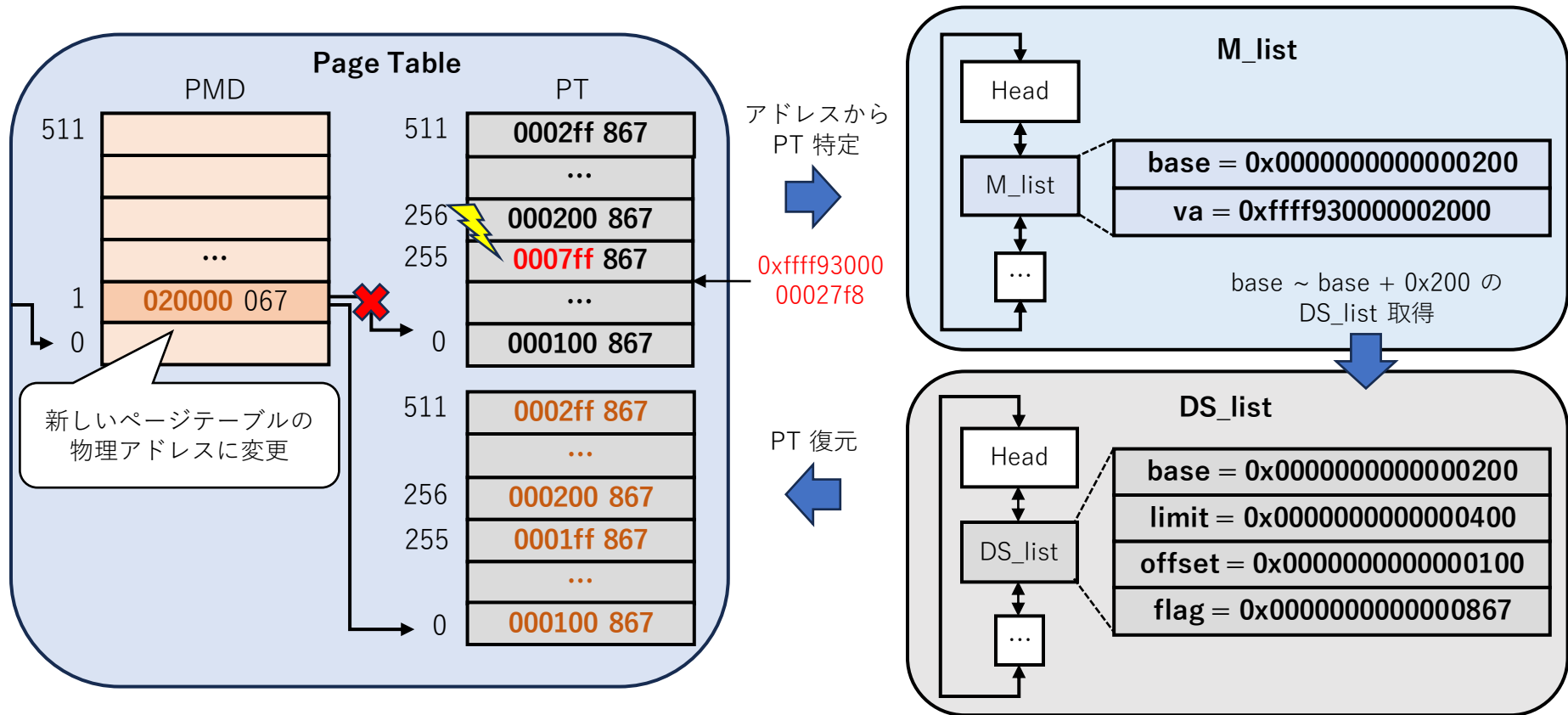
- 最下位ページテーブルを圧縮した復元用のメタデータのデータ構造
 - *Direct Segment* [Bas+, ISCA'13] をページテーブルの復元に応用
 - 仮想先頭アドレス *BASE*, 仮想末端アドレス *LIMIT*, 変換オフセット *OFFSET*, ページテーブルフラグ値 *FLAG* の 4 つの値を保持するリスト構造体
 - 連続する PTE が連続する物理ページに変換し, フラグ値が同じ場合, 一つに圧縮
 - 複数の PTE の情報を一つのデータ構造に圧縮することで, 低メモリスペースオーバーヘッドを実現



Recovery of Damaged Page Table

12

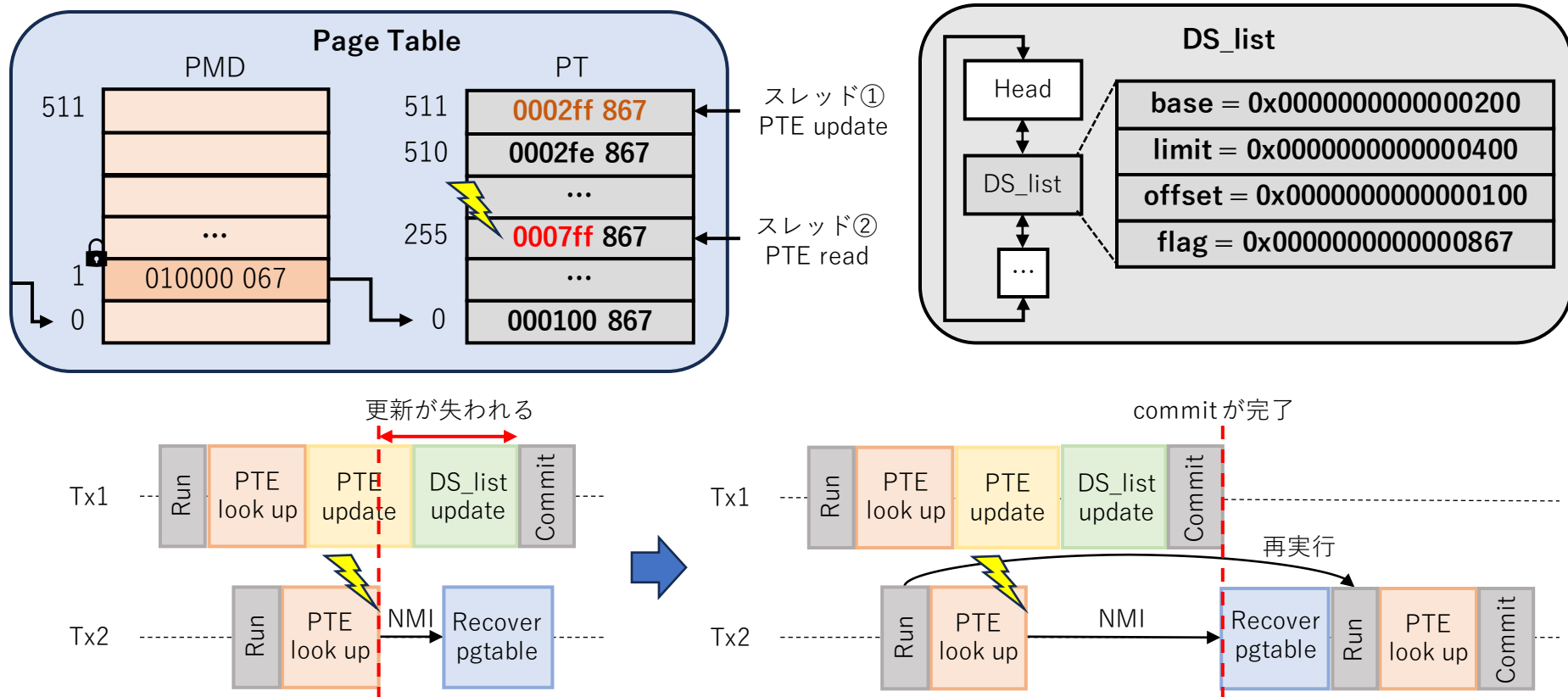
- M_list, DS_list を用いたページテーブルの復元手順
 - ① ECC モジュールにより通知されたアドレスを取得する
 - ② M_list からページテーブルを特定し、ユーザ仮想アドレスを得る
 - ③ ユーザ仮想アドレスに対応するページテーブルの範囲内の DS_list を取得する
 - ④ ページテーブルを新しく確保し、DS_list から情報を復元する
 - ⑤ 上位のアドレスを変更し、復元したページテーブルに差し替える



Handling Multi-threaded PTE Operations

13

- PTE の更新, 読み込みは PT 単位のトランザクションにてログを管理
 - DS_list が更新される前に PT が復元され, 整合性が取れない可能性がある
 - EMEs を検知した場合, PMD にてロックを取得し, PT へのアクセスを禁止
 - EMEs を検知したスレッド以外は commit が完了するまで実行
 - EMEs を検知したスレッドは NMI を発行し, 全処理の commit を確認した後, PT の復元を実行し, PTE アクセスから再実行



- 実装
 - Linux 6.1.35 上に実装
 - mmap, munmap によるページテーブルの更新から M_list, DS_list を同期する機構を実装
 - M_list, DS_list からページテーブルを復元する機構を実装
- 実験環境
 - OS : Ubuntu 22.04
 - マシン : Intel(R) Xeon(R) Gold 5218 2.30 GHz, 4 core, 40 GB RAM

- 目的
 - 提案手法が EMEs によってダメージを受けたページテーブルを適切に復元することができるかを確認する
- 方法
 - フォールトを模したシステムコールを発行し, mmap にて作成した M_list, DS_list を用いてページテーブルを復元する
 - ページテーブルが適切に復元できているかを確認する
- 結果
 - フォールトを挿入したすべてのページテーブルの復元が成功した

EXP2 : DS_list Memory Size

16

- 目的

- 提案手法が低メモリスペースオーバーヘッドであるかを確認する

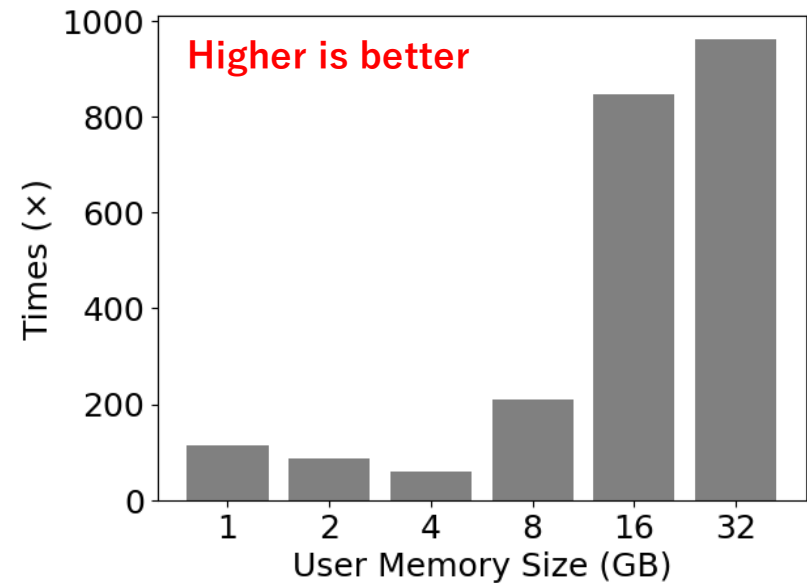
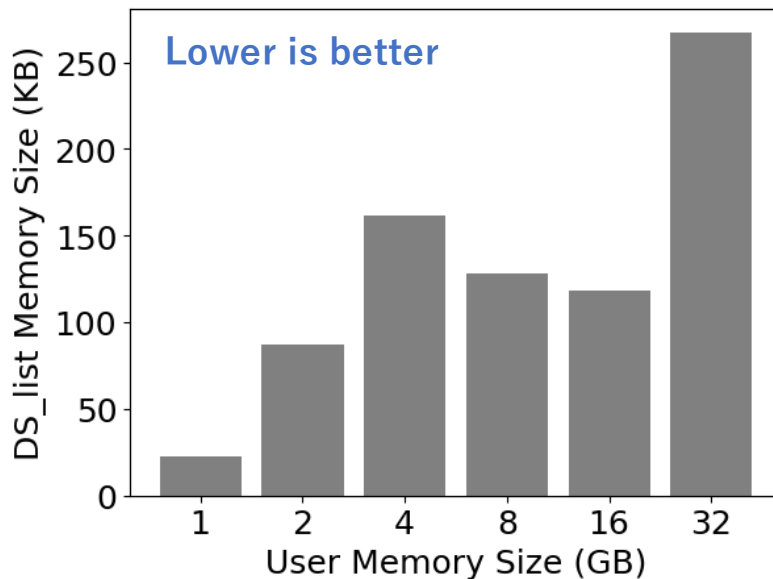
- 方法

- ユーザにおける割り当てメモリサイズを 1, 2, 4, 8, 16, 32 GB と変化させ、作成された DS_list のメモリサイズを計測
- ページテーブルの複製を保持する場合のメモリサイズと比較し、メモリサイズが何倍小さくなったかを計測
 - DS_list のメモリサイズ : DS_list のノード数 $\times$ 48 B
 - ページテーブルの複製のメモリサイズ : 割り当て PT 数 $\times$ 4 KB

EXP2 : Result

17

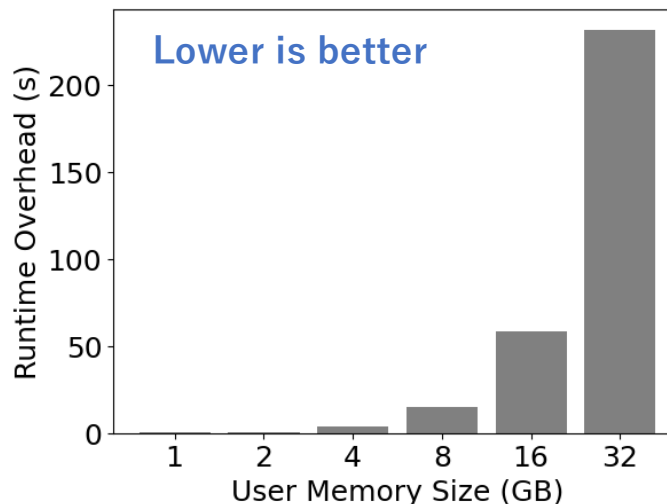
- 提案手法は低メモリスペースオーバーヘッドであった
 - DS_list のサイズは割り当てメモリサイズにつれ増加したが、ページテーブルのメモリサイズに比べてかなり圧縮できている
 - 特にユーザにて 32 GB のメモリが割り当てられている場合、DS_list のメモリサイズは約 267 KB、比較して約 962 倍小さくなった
 - DS_list のメモリサイズは PT の状況によりかなり変化する
 - 割り当てメモリサイズ 1 GB における DS_list の最悪値は約 54 KB
 - 割り当てメモリサイズ 32 GB における DS_list の最良値は約 23 KB



EXP3 : Runtime Overhead

18

- 目的：M_list, DS_list の作成時のランタイムオーバーヘッドの計測
- 方法
 - 割り当てメモリサイズを 1, 2, 4, 8, 16, 32 GB と変化させ、M_list, DS_list の作成時間を計測
- 結果
 - ランタイムオーバーヘッドはメモリサイズに指数関数的に比例した
 - 割り当てメモリサイズが 1, 2 GB の場合、オーバーヘッドは 1 秒未満
 - 割り当てメモリサイズが 32 GB の場合、オーバーヘッドは約 231 秒
 - 最適化の余地あり
 - 多くのインメモリ App. は起動時にメモリ割り当てを行うため運用時にはこのオーバーヘッドは生じない

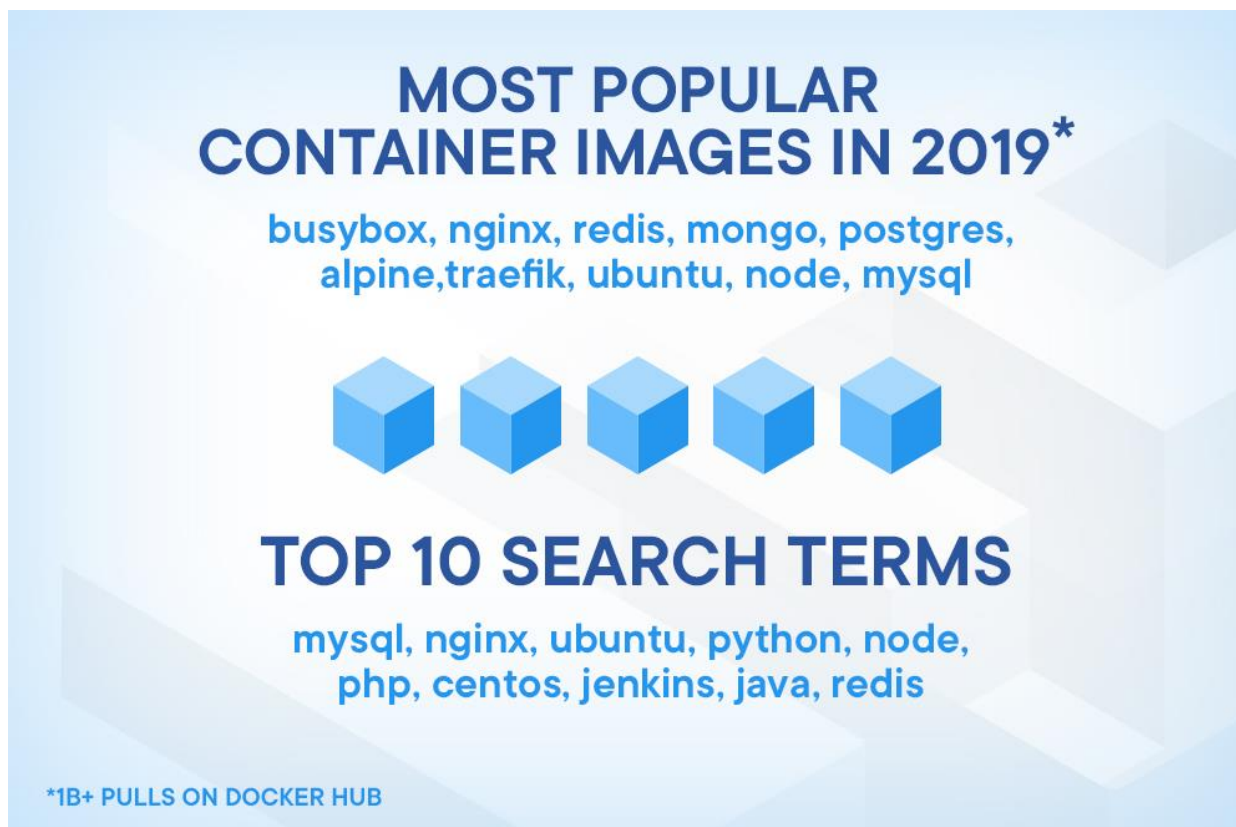


- まとめ
 - EMEs への耐性を有するページテーブル管理機構を提案した
 - 提案手法の実現のため、M_list, DS_list の 2 つのデータ構造を導入し、これらを用いたページテーブルの復元機構を導入した
 - 提案手法を Linux 6.1.35 に実装・実験を行った結果、低メモリスペースオーバーヘッドでページテーブルの復元に成功した
- 今後の課題
 - PTE のフラグ値を変更する処理 (mprotect, swap, etc.) への対応
 - 最下位以外のページテーブルの復元機構の設計
 - Real-world なアプリケーションでの実験

補足：Docker Hub DL Ranking

20

- Docker Hub で最も人気のあるコンテナイメージのトップ 10
 - Busybox, Nginx, Redis, MongoDB, PostgreSQL, Alpine, Traefik, Ubuntu, Node.js, MySQL
 - インメモリ DB などの Memory-intensive な App. が多い



補足：ページテーブル割り当て数

21

- 最下位ページテーブル PT はかなり数が多いが、
最下位以外のページテーブルはそれほど数が多い

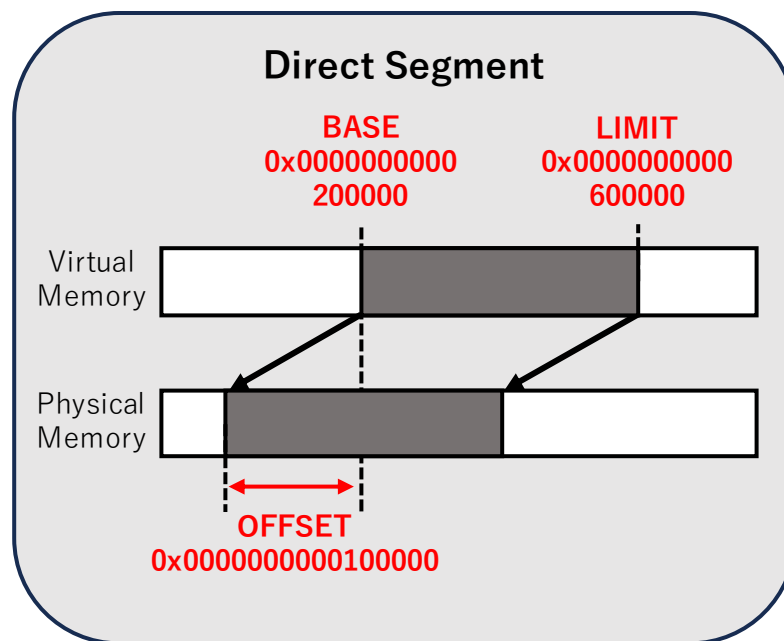
⇒ 最下位以外のページテーブルは複製を保持することで復元

	PGD	PUD	PMD	PT
1 GB	1	2	4	519
2 GB	1	2	5	1031
4 GB	1	2	7	2054
8 GB	1	2	11	4102
16 GB	1	2	19	8199
32 GB	1	2	35	16390

補足：Direct Segment

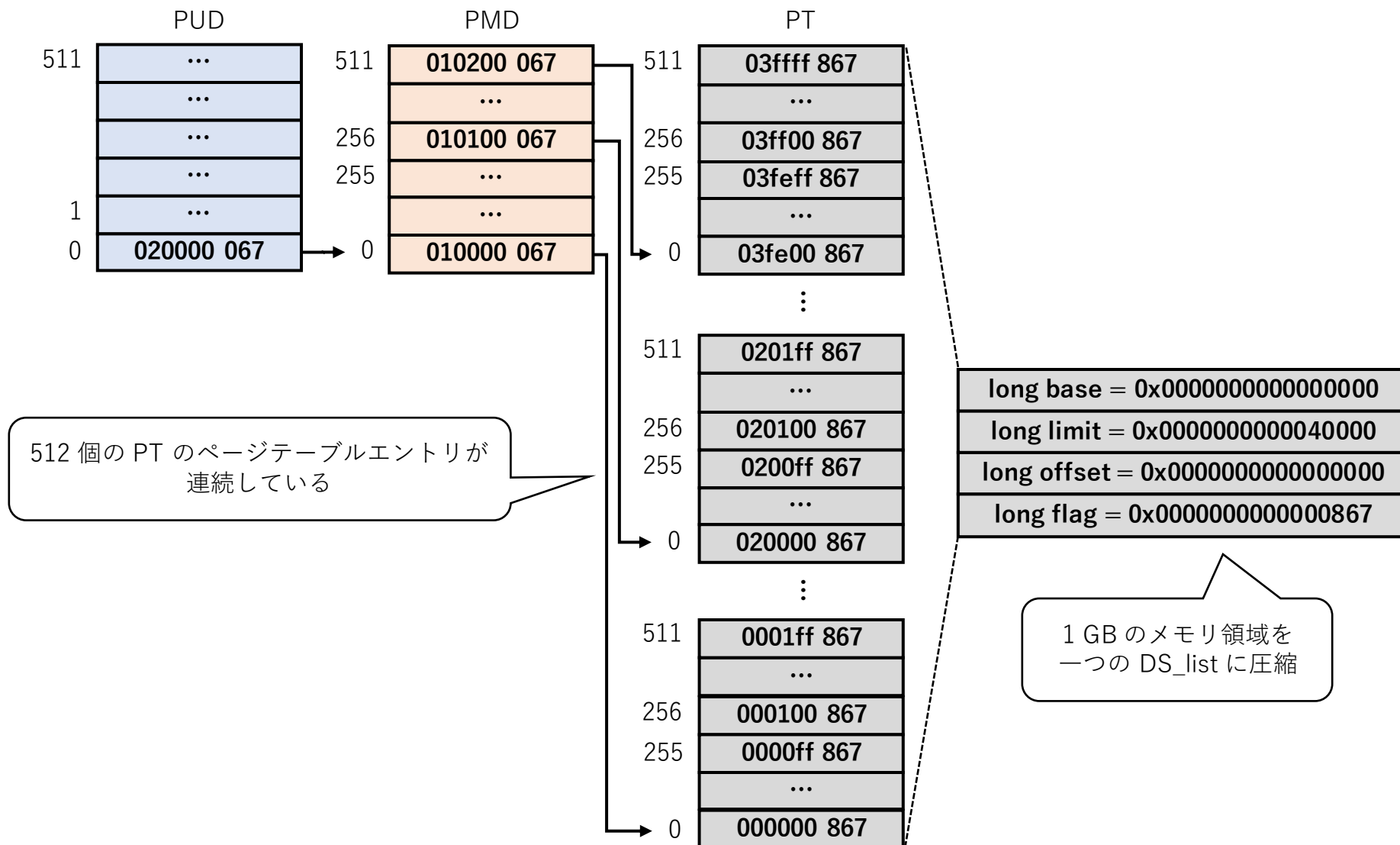
22

- 連続するメモリマッピングをセグメントで表すハードウェア機構 [Bas+, ISCA'13]
 - 仮想メモリ領域の先頭アドレスを *BASE*, 末端アドレスを *LIMIT*, 仮想アドレスと物理アドレスの差分を *OFFSET* として保持
 - $BASE \leq VA < LIMIT$ のとき, $PA = VA - OFFSET$ として計算可能



補足：Best Case of DS_list

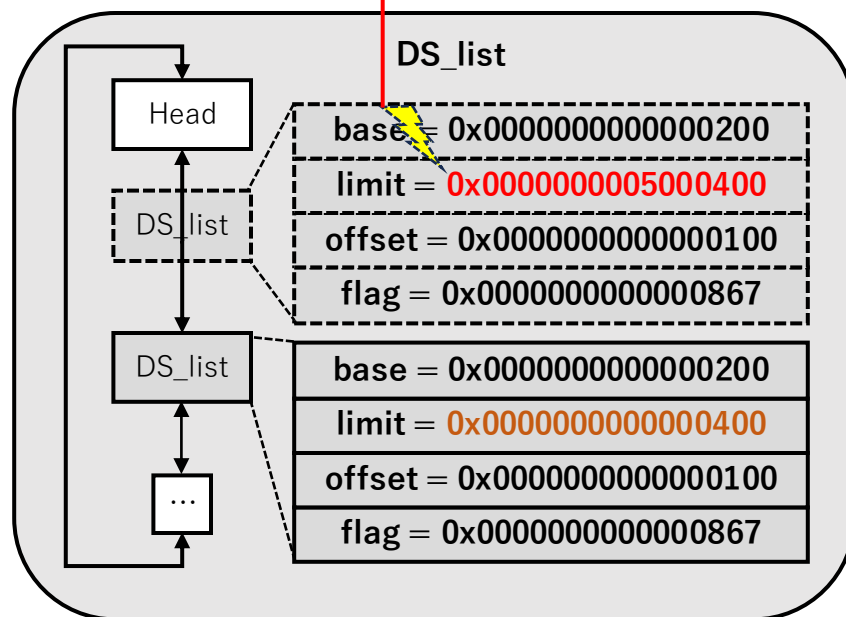
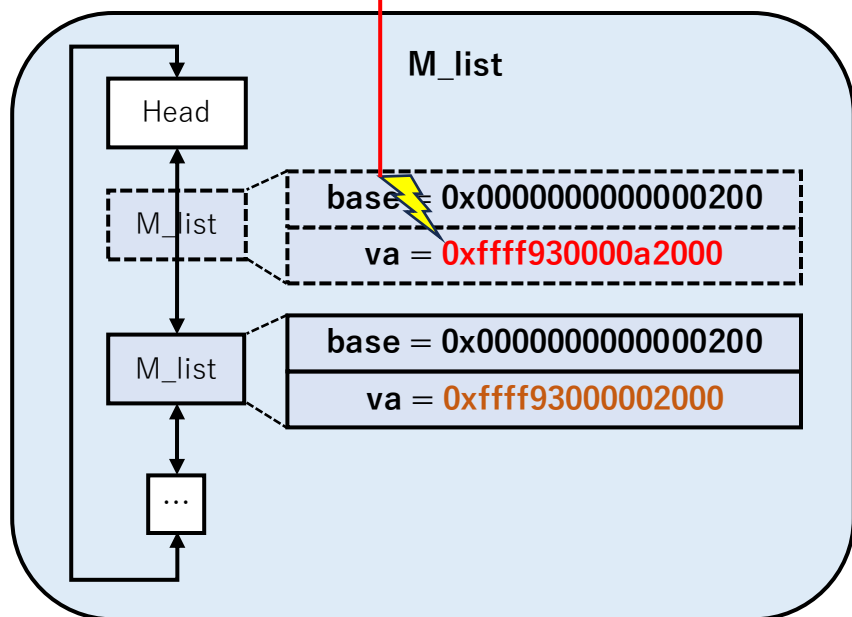
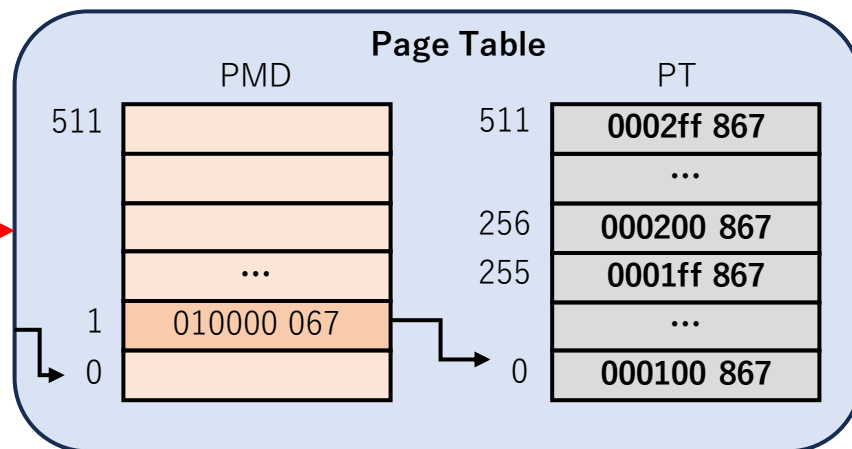
23



補足 : EMEs in M_list, DS_list

24

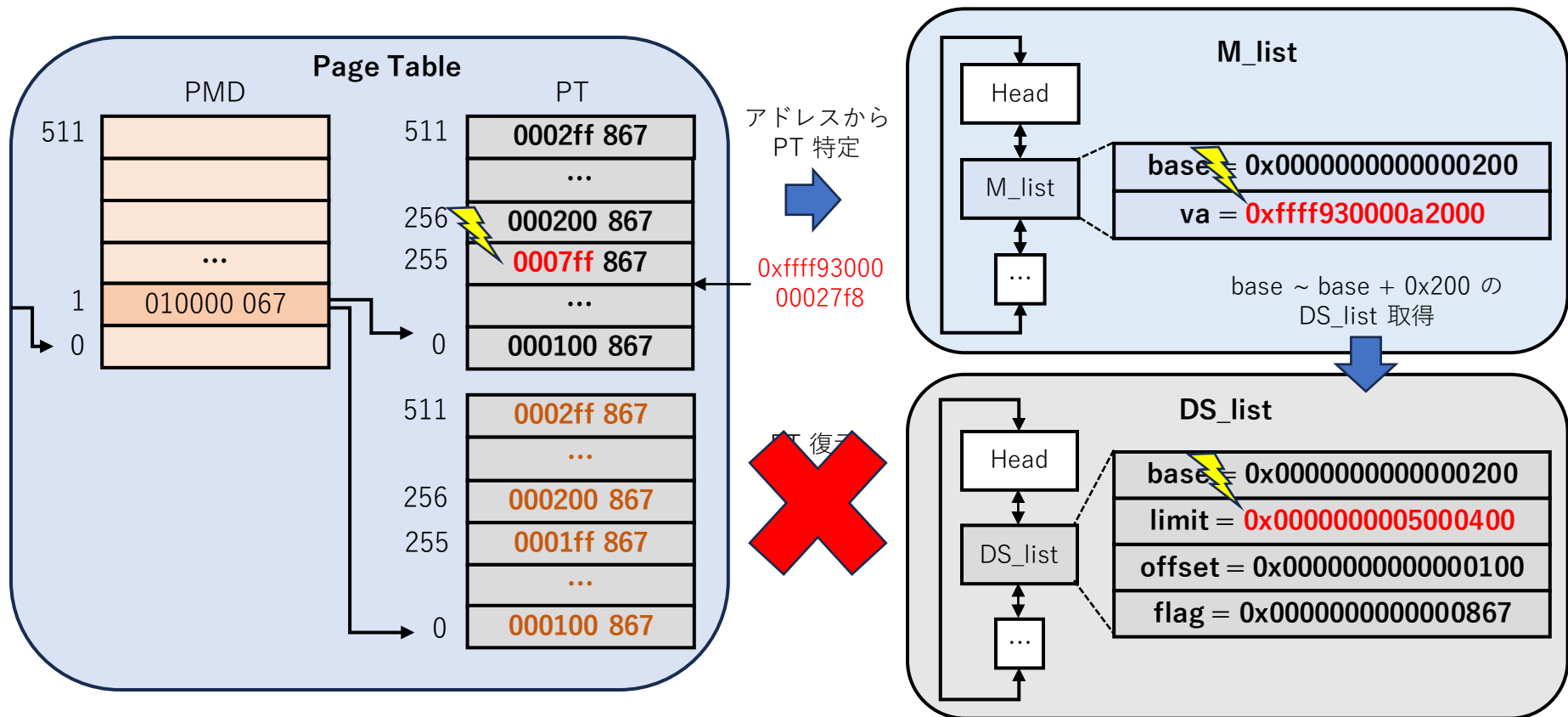
ページテーブルを探索し、
破損した M_list, DS_list の
情報を復元



補足：Worst Case (1)

25

- PTE と対応する M_list or DS_list のどちらも EMEs が発生した場合
⇒ Worst Case であり，OS の再起動を実行する

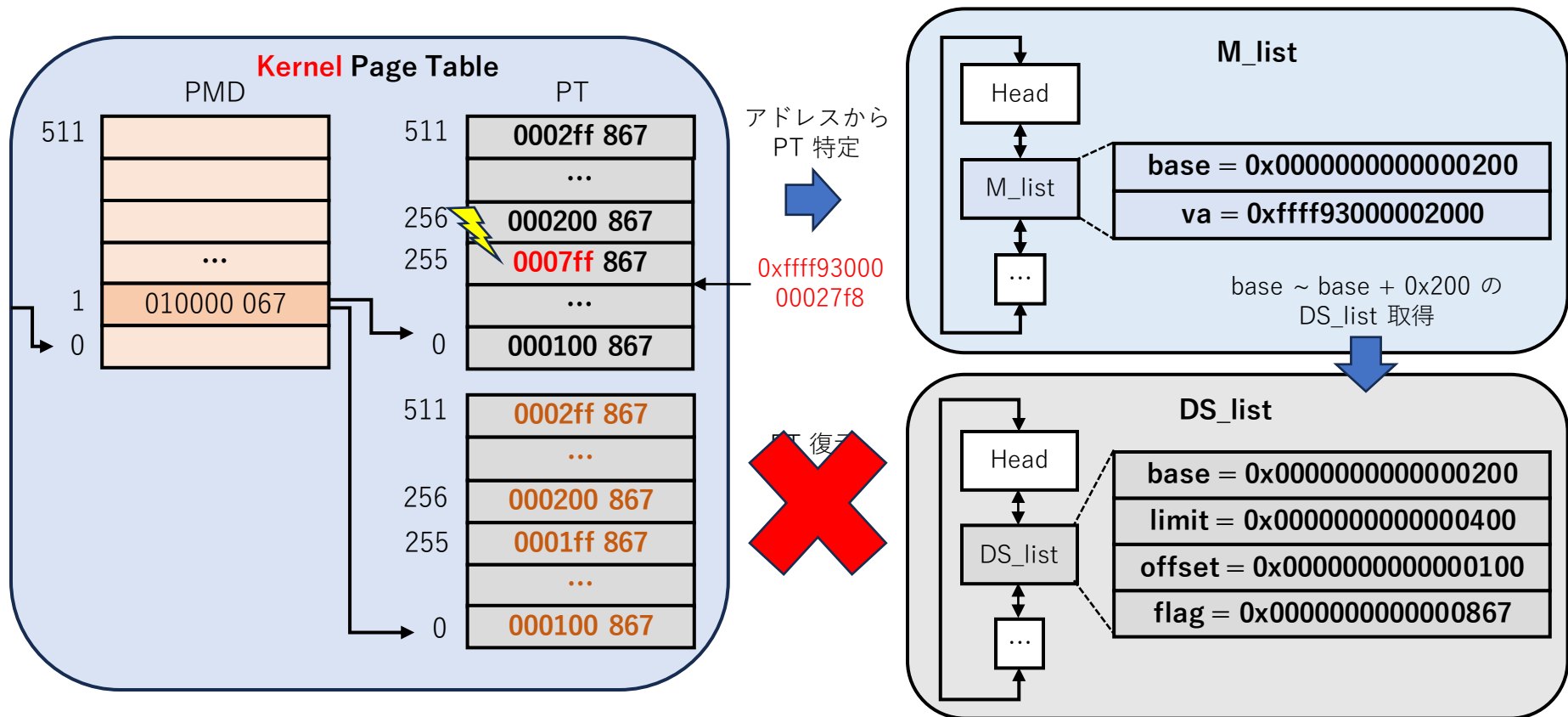


補足：Worst Case (2)

26

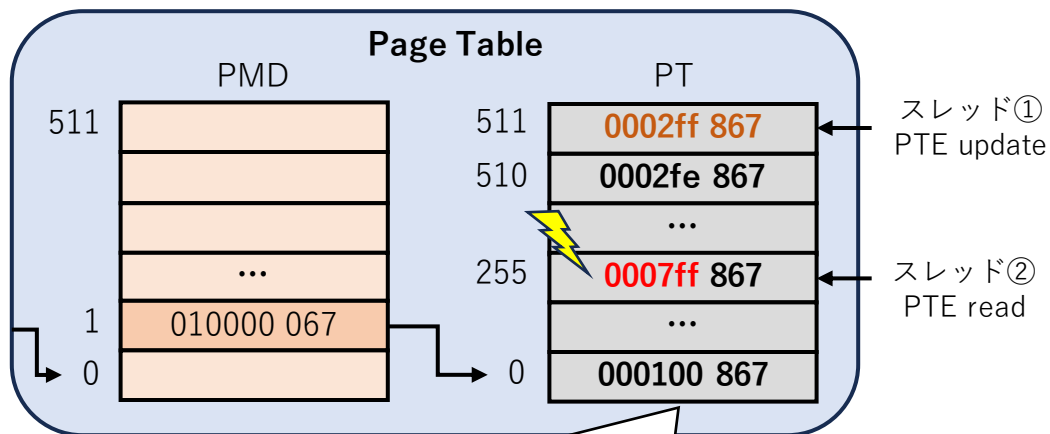
- カーネルのページテーブルの PTE にて EMEs が発生し、その PTE が M_list or DS_list のアクセスに必要な場合

⇒ Worst Case であり、OS の再起動を実行する

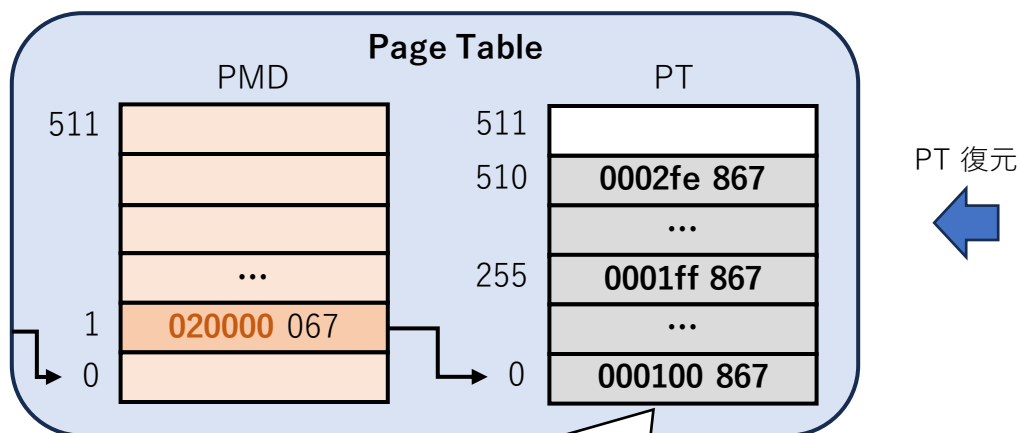
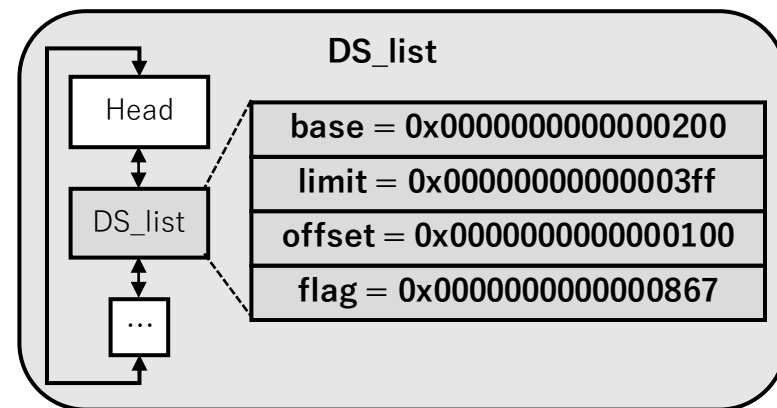


補足 : Bad Case in Multi-thread

27



スレッド①がDS_listを更新する前に
スレッド②がEMEsを検知し、PT復元



スレッド①の更新が反映されず、
復元されてしまう

